

UI-CS-AI-2026-401628201-PRJ4

پروژه شماره ۳: جستجوی خصمانه (۱۴۰۵/۰۴/۱۴)

[مهلت انجام: ۱۴۰۵/۰۴/۳۱]

آنچه باید پاسخ دهید:

بازی مین روب



مین روب یا Minesweeper، یک بازی قدیمی ویندوزی که بعضی‌ها را هم‌زمان عصبانی و معتاد کرد! شاید شما هم خاطره‌ای از لرزیدن دستتان هنگام کلیک روی مربع‌های خاکستری را به یاد داشته باشید. اما این بار نه با حدس و خطا، بلکه با منطق و استدلال قرار است این بازی را حل کنیم.

هدف این پروژه این است که نشان دهیم چگونه می‌توان قوانین ساده یک بازی را به صورت حقایق و قواعد منطقی بیان کرد و سپس با یک موتور استنتاج، راه حل را پیدا نمود.

۱. پیش‌نیازها

کتابخانه‌ی پیش‌نیاز در بخش ۳ معرفی شده‌است.

از این [لینک](#) می‌توانید فایل‌های محیط را دانلود کنید.

۲. محیط

در بازی مین روب دو کنش (Action) وجود دارد:

- **Reveal (فاش کردن):** با کلیک چپ روی هر مربع، آن خانه فاش می‌شود. اگر مین (Mine) باشد شما می‌بازید! در غیر این صورت اگر خانه امن باشد، یک سرنخ (Clue) افشا می‌شود. هر سرنخ شامل یک عدد بین ۰ تا ۸ است و بیانگر این است که در همسایگی این خانه چند مین وجود دارد.
 - **Flag (پرچم گذاری):** با کلیک راست روی هر مربع، آن خانه به عنوان مین شناخته می‌شود. می‌توان خانه‌ای که مین نیست را پرچم گذاری کرد، اما این باعث تصمیم‌گیری اشتباه در آینده می‌شود که ممکن است برایتان گران تمام شود.
- شرط برد:** وضعیتی است که تمام خانه‌های مین پرچم گذاری شده و تمام خانه‌های امن فاش شوند. می‌توانید در [این لینک](#) روند حل یک نمونه از این بازی را مشاهده کنید.

۳. منطق درجه اول

منطق درجه اول یا First-Order Logic (FOL) زبانی است که به ما اجازه می‌دهد جهان را نه با متغیرهای ساده، بلکه با «اشیاء» و «روابط» بین آن‌ها توصیف کنیم. در پایتون، کتابخانه pyDatalog این امکان را فراهم می‌کند که مستقیماً از پارادایم برنامه‌نویسی منطقی (شبیه به Prolog) استفاده کنیم.

نصب و راه‌اندازی

برای شروع، ابتدا باید کتابخانه pyDatalog را نصب کنید.

مفاهیم پایه: ترم، فکت، قانون و کوئری

برای استفاده از pyDatalog، باید چهار مفهوم کلیدی را درک کنید:

- **ترم (Term):** متغیرها یا اشیاء (مثل X ، Y یا اعداد یا مسندها).
- **فکت (Fact):** حقایقی که در مورد جهان می‌دانیم (داده‌های اولیه).
- **قانون (Rule):** روابط منطقی که دانش جدید تولید می‌کند (اگر ... آنگاه ...).
- **کوئری (Query):** پرسش از سیستم برای یافتن جواب.

یک مثال ساده

فرض کنید می‌خواهیم رابطه پدری و پدربزرگی را تعریف کنیم:

```

1 from pyDatalog import pyDatalog
2
3 # 1. Define Terms (Variables)
4 pyDatalog.create_terms('X, Y, Z, Father, GrandFather')
5
6 # 2. Add Facts (Father(Parent, Child))
7 + Father('Ali', 'Reza')      # Ali is father of Reza
8 + Father('Reza', 'Sara')     # Reza is father of Sara
9
10 # 3. Define Rules (Logic)
11 # Logic: X is GrandFather of Y IF X is Father of Z AND Z is Father of Y
12 GrandFather(X, Y) <= Father(X, Z) & Father(Z, Y)
13
14 # 4. Query
15 print(list(GrandFather(X, 'Sara')))
16 # Output: [('Ali',)]
    
```

در مثال بالا، عملگر $=$ به معنی «اگر» و عملگر $\&$ به معنی «و» منطقی است. این قانون معادل عبارت ریاضی زیر است:

$$\forall X \forall Y \left(\exists Z (Father(X, Z) \wedge Father(Z, Y)) \rightarrow GrandFather(X, Y) \right)$$

یادگیری سینتکس

کتابخانه pyDatalog ساده‌ترین سینتکس را بین موتورهای استنتاج FOL دارد. برای یادگیری بیشتر می‌توانید از منابع زیر استفاده کنید:

- آموزش خلاصه این کتابخانه به همراه مثال: [Online Datalog Tutorial](#)
- یک مثال کاربردی از این کتابخانه: [Vishok's Blog on pyDatalog](#)
- یادگیری کامل سینتکس و قابلیت‌های پیشرفته: [pyDatalog Home](#)
- حل چند مثال دیگر با این کتابخانه: [pyDatalog Examples on GitHub](#)

موتورهای استنتاج دیگر

موتورهای دیگری هم وجود دارند که می‌توانند حقایق را دریافت کنند و بر اساس منطق مرتبه اول مسائل را حل کنند. یکی از مشهورترین این موتورها SWI-Prolog است. در صورت تمایل، می‌توانید کد خود را به این زبان نیز بنویسید.

```

1 % Define facts
2 father(ali, reza).
3 father(reza, sara).
4
5 % Define rule: X is grandfather of Y if X is father of Z and Z is father of Y
6 grandfather(X, Y) :- father(X, Z), father(Z, Y).
7
8 % Query examples
9 ?- grandfather(X, sara).
10 % Output: X = ali
    
```

سپس به کمک کتابخانه‌ی pyswip که یک Wrapper برای این موتور استنتاج است، می‌توانید از قواعد نوشته‌شده در پایتون استفاده کرده و کوئری بزنید.

- دانلود موتور استنتاج SWI-Prolog: swi-prolog.org

- آموزش کار با کتابخانه PySwip: pyswip.org

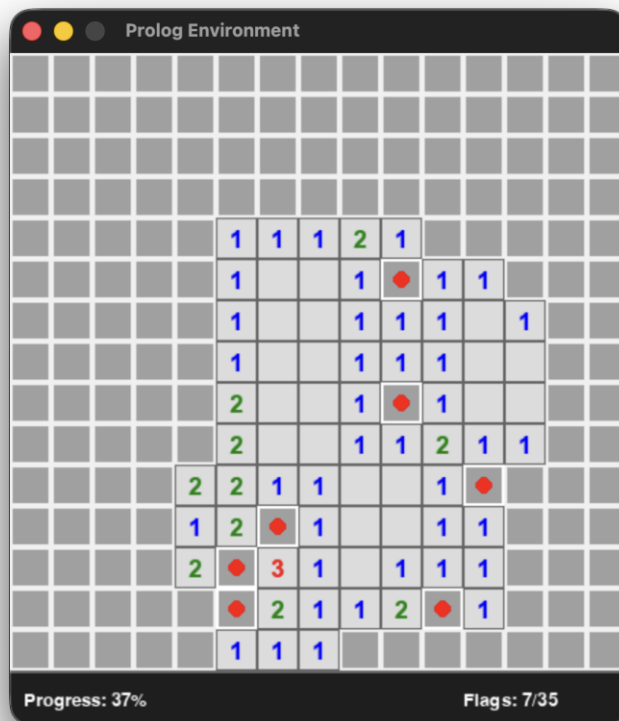
۴. آشنایی با محیط و توابع رابط

فایل محیط شامل کلاس Minesweeper در اختیار شما قرار گرفته است. شما نباید کدهای داخلی این کلاس را تغییر دهید، بلکه باید از توابع تعبیه شده (رابط‌ها) برای دریافت اطلاعات و ارسال دستورات استفاده کنید.

محیط

برای ساخت محیط، یک نمونه از کلاس Minesweeper می‌سازید. این کلاس شامل ویژگی‌های زیر است:

- ابعاد نقشه شامل تعداد سطر و ستون.
- تعداد مین در نقشه.
- وضعیت تصادفی بازی (Seed) برای بازتولید نقشه.



شکل ۱: مثالی از صفحه بازی ۱۵*۱۵ با $seed=42$, $mines=35$

راهنمایی: می‌توانید با استفاده از کد `manual.py` بازی را به صورت دستی بازی کنید. کلیک چپ معادل فاش کردن (Reveal) و کلیک راست معادل نشانه‌گذاری / عدم نشانه‌گذاری (Flag/Unflag) است.

دریافت حقایق (Facts)

در ابتدای بازی، شما می‌توانید با صدا زدن تابع `get_start_pos()` به مختصات خانه شروع برسید. تضمین می‌شود خانه شروع همیشه دارای سرنخ ۰ است (یعنی هیچ مینی اطراف آن نیست).

- **نکته مهم:** در مورد سایر حقایق همچون همسایه بودن یا نبودن، نشانه‌گذاری شده بودن یا نبودن، فاش شده بودن یا نبودن و سرنخ خانه‌های فاش شده، خودتان باید قانون (Rule) بنویسید.

کنش‌ها (Actions)

پس از اینکه موتور استنتاج شما تصمیم گرفت کدام خانه امن است یا کدام خانه مین است، باید با توابع زیر محیط بازی را آپدیت کنید:

- $reveal(r, c)$: خانه مشخص شده را فاش می‌کند. اگر خانه حاوی مین باشد، بازی تمام می‌شود.
 - خروجی: اگر خروجی بمب باشد مقدار ۱- و در غیر این صورت عددی بین ۰ تا ۸ خروجی می‌دهد که نشان‌دهنده سرخ آن خانه است. شما باید این حقیقت جدید را به مجموعه حقایق (Knowledge Base) خود اضافه کنید.
- $flag(r, c)$: روی خانه مشخص شده پرچم می‌گذارد (نشانه‌گذاری مین).
- $unflag(r, c)$: از روی خانه مشخص شده پرچم را حذف می‌کند.

۵. پیاده‌سازی

برای انجام موفق این پروژه، پیشنهاد می‌شود روند زیر را در اسکریپت اصلی خود (main.py) طی کنید:

آماده‌سازی اولیه

کلاس محیط را Import کنید و یک نمونه از بازی بسازید. ترجیحاً در حین توسعه و دیباگ کردن از یک Seed ثابت استفاده کنید تا نقشه تغییر نکند و نتایج قابل تکرار باشند:

```
ms = Minesweeper(rows=10, cols=10, mines=10, seed=42)
```

نوشتن قوانین و حقایق اولیه

شما باید حقایق اولیه بازی را به پایگاه دانش خود اضافه کنید. این حقایق می‌تواند شامل تعریف سلول‌ها، وضعیت خانه شروع و همسایه بودن یک سلول با سلول دیگر مانند $(R_1, C_1), (R_2, C_2)$ باشد. همچنین باید قوانینی مثل امن بودن یا مین بودن خانه‌ها را بر اساس منطق درجه اول تعریف کنید:

- **قانون امنیت (Safe Rule):** اگر یک خانه باز شده دارای عدد N باشد و تعداد همسایه‌های پرچم‌گذاری شده‌ی آن دقیقاً برابر با N باشد، تمام همسایه‌های باز نشده‌ی دیگر آن خانه، امن هستند.
- **قانون خطر (Mine Rule):** اگر یک خانه باز شده دارای عدد N باشد و تعداد کل همسایه‌های باز نشده (به علاوه پرچم‌ها) برابر با N باشد، تمام همسایه‌های باز نشده‌ی آن قطعاً مین هستند.

نکته: قوانین و استنتاج‌های پیچیده‌تری نیز در بازی مین‌روب وجود دارند که استخراج و پیاده‌سازی آن‌ها به عهده شما واگذار شده است.

حلقه اصلی بازی (Game Loop)

شما نیاز به یک حلقه while دارید که تا زمان برقرار بودن شرط `ms.game_over == False` به تکامل خود ادامه دهد. در هر تکرار از این حلقه، مراحل زیر باید به ترتیب اجرا شوند:

۱. به‌روزرسانی و استخراج حقایق این دور: به دست آوردن تعداد خانه‌های پنهان (Hidden) یا پرچم‌گذاری شده (Flagged) که در اطراف خانه‌های فاش شده قرار دارند.

۲. پرس‌وجو (Query): از موتور استنتاج منطقی بپرسید «کدام خانه‌ها امن هستند؟» (`Safe(R, C)`) و «کدام خانه‌ها مین هستند؟» (`Mine(R, C)`).

۳. اعمال تغییرات (Actions): نتایج به دست آمده از کوئری‌ها را با استفاده از توابع `reveal(r, c)` و `flag(r, c)` روی محیط بازی اعمال کنید. اگر در یک دور کامل، هیچ حرکت منطقی و قطعی پیدا نشد، عامل هوشمند شما می‌تواند یک خانه را به صورت تصادفی انتخاب کند (ریسک) یا بازی را متوقف کند.

۴. اضافه کردن حقایق جدید: سرنخ (عدد) خانه‌های جدید فاش شده و مختصات خانه‌های پرچم‌گذاری شده را به مجموعه دانش (Knowledge Base) بازی اضافه کنید.

نمایش (Render): تابع `ms.render()` را صدا بزنید تا آخرین وضعیت گرافیکی یا متنی صفحه بازی را مشاهده کنید.

۶. نحوه ارزیابی

عامل هوشمند شما باید بتواند در ابعاد و سختی‌های مختلف، بازی را حل کند. برای تحویل پروژه، کد شما روی سناریوهای دیگری نیز تست خواهد شد. برای ارائه سعی کنید کدتان را برای موارد مشخص شده در جدول زیر بهینه کنید:

سطح	ابعاد (Rows * Cols)	تعداد مین	Seed	انتظار
ساده (تست منطق)	۹*۹	۹	۹۹	حل کامل بدون باخت
استاندارد	۱۵*۱۵	۳۵	۴۲	حل کامل بدون باخت

چالشی	۲۰*۲۰	۷۵	۱۲۳	حداقل ۸۰٪ پیشرفت
بزرگ (تست سرعت)	۴۵*۸۰	۲۰۰	-۱	عدم کرش کردن برنامه

جدول ۱: سناریوهای ارزیابی پروژه

نکته مهم: در برخی نقشه‌ها ممکن است موقعیت‌هایی پیش بیاید که هیچ استنتاج منطقی قطعی وجود نداشته باشد (شانس -۵۰). در این موارد، باختن عامل ایرادی ندارد، اما عامل باید تا حد ممکن با استفاده از منطق پیشروی کرده باشد.

آنچه باید تحویل دهید:

- کد پروژه
 - گزارش از پروژه به صورت pdf (استفاده از گیت اجباری است. لینک پابلیک ریپازیتوری خود را در این گزارش بگذارید)
- را در سامانه کوئرا بارگزاری کنید. (یک نفر از هر گروه کافی است).

هشدار: لطفاً انجام پروژه را به روز پایان ددلاین موکول نکنید!

تندرست و پیروز باشید

فریاد نصیری منعم